

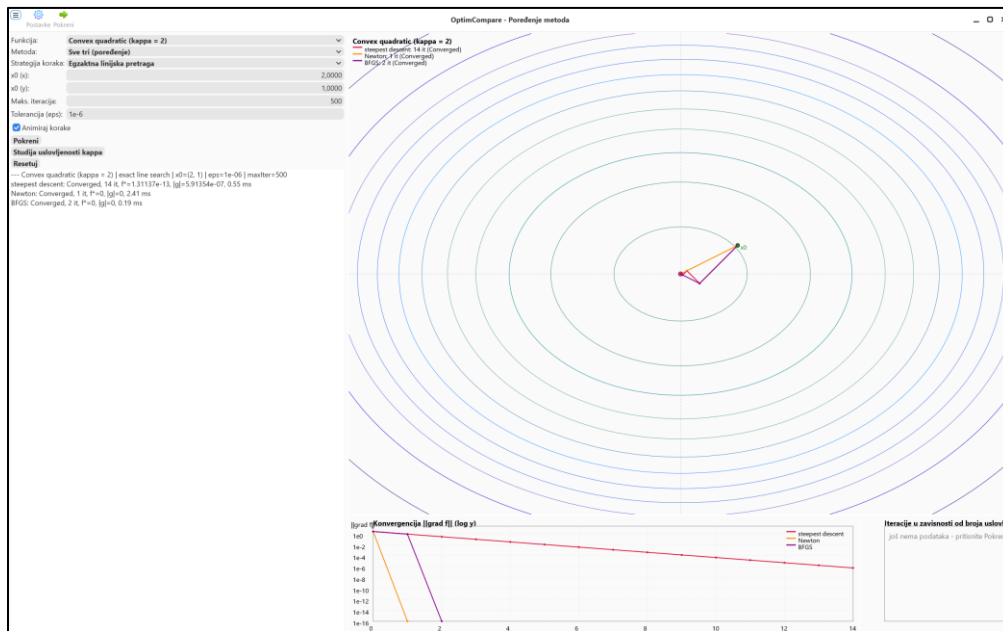
Comparative Analysis of Three Optimization Methods - Steepest Descent , Newton's Method , BFGS

1. Introduction

In this project I look at three ways to find the minimum of a function: steepest descent, Newton's method, and BFGS. They all do the same thing, so they start at some point and take steps to move closer to the minimum. The step rule is always the same: $x_{k+1} = x_k + \alpha_k d_k$. Here x_k is where we are right now, d_k is the direction to move in, and α_k is how far to move.

The only real difference between the three methods is how each one picks the direction d_k . Steepest descent only uses the gradient. That's a first-order method, it just looks at which way is "downhill" right now. Newton also uses the Hessian (the matrix of second derivatives), which tells about curvature, so how the slope itself is changing. That makes it a second-order method, and it lets Newton "see" the shape, not just which way is down. BFGS is in between, it doesn't use the real Hessian, but it keeps building a guess of it as it goes, based on how the gradient changed after each step.

To compare the three methods, in the app we can pick a test function, pick a method (or all three at once), and pick how big the steps should be. Then we can watch the paths on a contour plot and see how fast each method gets to the minimum. There is also a mode that tests each method on harder and harder functions, to see how they slow down (or don't).



2. Methodology

2.1 The three methods

All three methods use the exact same code structure underneath. There's one shared loop that runs the iterations, and the only thing that changes between methods is how the direction d is computed inside that loop:

Method	Search direction	Notes / safeguards
Steepest Descent	$d = -g$	Only uses the gradient. Works fine most of the time, but gets slow on hard functions

Method	Search direction	Notes / safeguards
Newton	Solve $H d = -g$	Uses the real Hessian to pick the direction. Very fast when it works. If the Hessian looks wrong (saddle point), it just uses gradient descent for that step instead, so it never really breaks
BFGS	$d = -B g$ (B = inverse-Hessian approx.)	Builds a guess of the Hessian as it goes, instead of using the real one. Skips an update if it would make the guess worse

A bit more detail on how each direction is actually computed in the code:

- Steepest descent just sets $d = -g$, so it always moves straight down the current slope.
- Newton solves the linear system $H \cdot d = -g$ for d , using a dense matrix solver. This means it needs to build and solve with the full Hessian matrix at every single step, which is more work per step, but usually needs far fewer steps.
- BFGS keeps a matrix B that approximates the inverse Hessian, and sets $d = -B \cdot g$. After each step, B is updated using the difference between the old and new gradient (standard BFGS update formula), so B slowly gets more accurate the more steps it takes.

2.2 How the step size is chosen

Each method also needs to decide how far to move in the direction it picked. There are two ways to do this in the app, and both work the same way no matter which of the three methods is running:

- **Exact line search:** tries to find the best step size α , meaning the one that gets the function value as low as possible along that direction. For quadratic functions there is exact formula for this: $\alpha = -(g^T d) / (d^T H d)$. For other, curvier functions, there's no simple formula, so the app searches for the best step size numerically, by trying a growing range of step sizes and then narrowing down on the best one. So, we're walking in a straight line and we want to find the lowest point along that line, like finding the bottom of a valley.

First we do bracketing, we take small steps and keep doubling them, checking the function value each time. As long as the value keeps going down, we keep going. The moment it goes back up, we stop, because that means we just walked past the bottom. Now we know the real lowest point is somewhere between our last two steps. Then we do golden-section search to zoom in on that range. Instead of just checking the middle, we pick two points inside the range at specific spots. We check both, throw away the side with the higher value since the answer can't be there, and we're left with a smaller range. We keep doing this over and over, and the range keeps shrinking, until it's so tiny we've basically found the exact best step size

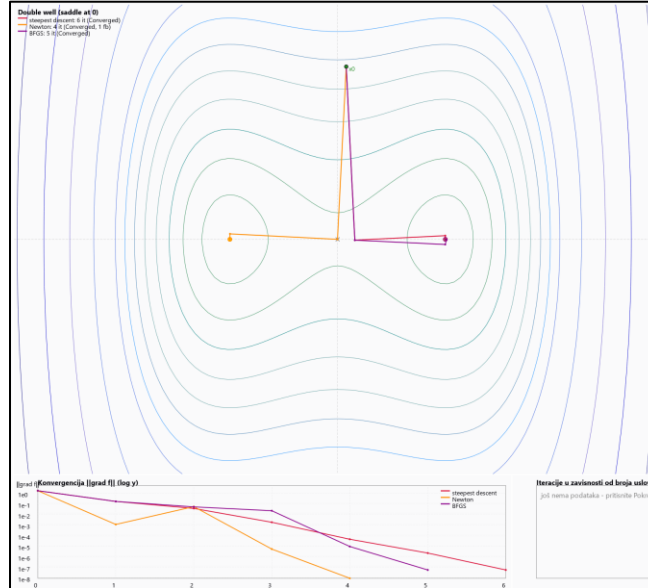
- **Armijo backtracking:** It starts with a step size of 1 and keeps cutting it in half, until the function value actually gets smaller by "enough" (this is a standard condition from optimization theory, using a small safety constant $c1 = 10^{-4}$). It doesn't try to find the perfect step, just a good-enough one, quickly. We start by trying the biggest reasonable step, a step size of 1. Then we check: did this step actually make the function value go down by enough, not just a tiny bit? If it did, we're happy and we take that step. If it didn't go down enough, we cut the step size in half and check again. We keep cutting it in half over and over until we find a step size that finally satisfies the condition. The condition we're checking is this formula: $f(x + \alpha d) \leq f(x) + c1 \cdot \alpha \cdot g^T d$

2.3 Test functions used

I tested all three methods on a set of 2D functions (functions of two variables), so I could draw everything on the contour plot. The functions are split into three groups, from easy to hard:

Group	Function(s)	Character	Purpose in the study
A	Convex quadratic ($\kappa=2$); strongly convex quad + exp	strongly convex	The easy ones. Just to check all three methods work and converge fast

Group	Function(s)	Character	Purpose in the study
B	Ill-conditioned quadratic ($\kappa=100$); rotated quadratic ($\kappa=2000$)	Elongated, narrow bowls	Harder. Steepest descent starts zig-zagging here, while Newton and BFGS handle it much better
C	Double well, Rosenbrock, Himmelblau	Non-convex, saddle points, curved valleys, multiple minima	The tricky ones, with saddle points and more than one minimum.



Function	Formula	Minimum at
Convex quadratic ($\kappa=2$)	$f(x,y) = 0.5 \cdot [x \ y] \cdot A \cdot [x \ y]^T$	(0, 0); $a_{11} = \cos^2(\text{angle}) + \kappa \cdot \sin^2(\text{angle})$ $a_{22} = \sin^2(\text{angle}) + \kappa \cdot \cos^2(\text{angle})$ $a_{12} = (1 - \kappa) \cdot \cos(\text{angle}) \cdot \sin(\text{angle})$
Strongly convex (quad + exp)	$f(x,y) = x^2 + y^2 + \exp(0.1(x+y))$	(-0.0495, -0.0495)
Ill-conditioned quadratic ($\kappa=100$)	same bowl form as above, just stretched more ($\kappa=100$) and tilted 15°	(0, 0)
Rotated quadratic ($\kappa=2000$)	same bowl form, stretched even more ($\kappa=2000$) and tilted 30°	(0, 0)
Double well	$f(x,y) = 0.25x^4 - 0.5x^2 + 0.5y^2$	(-1, 0) and (1, 0), saddle at (0, 0)
Rosenbrock	$f(x,y) = (1-x)^2 + 100(y-x^2)^2$	(1, 1)
Himmelblau	$f(x,y) = (x^2+y-11)^2 + (x+y^2-7)^2$	4 minima: (3,2), (-2.81,3.13), (-3.78,-3.28), (3.58,-1.85)

2.4 Keeping things from breaking

Newton and BFGS both use curvature information (the Hessian, or a guess of it). But that information can go wrong sometimes, especially when you're far from the minimum, or near a saddle point. So I added a safety check for each of them:

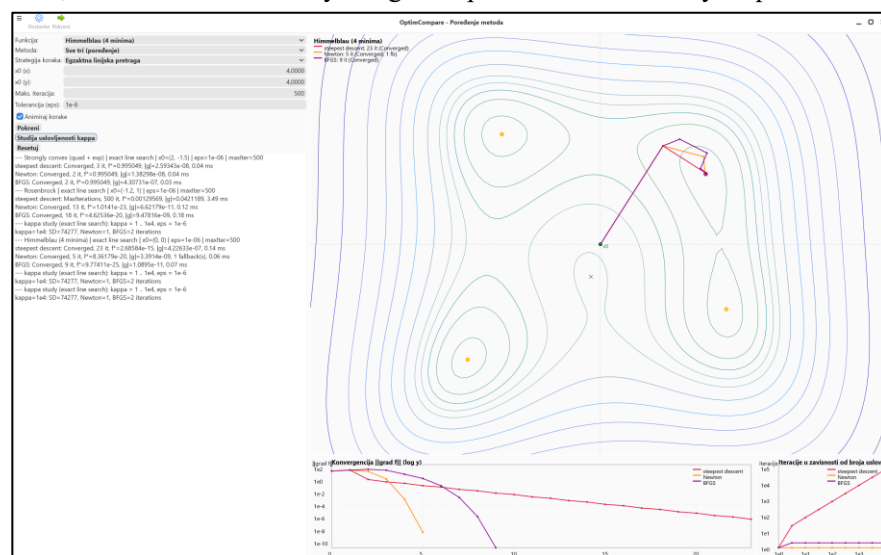
- Newton: before taking a Newton step, the app checks that the direction it got actually points downhill. If solving $H \cdot d = -g$ fails, or if the direction it finds does not actually decrease the function, the app just falls back to plain steepest descent ($d = -g$) for that one step.
- BFGS: before applying an update to the guessed matrix B, the app checks a simple condition called the curvature condition ($y^T s > 0$, where y is the change in gradient and s is the change in position). If an update would break this condition, meaning it would actually make the guess worse, the app just skips that update and keeps the old B. If B ever becomes numerically unstable, it gets reset back to the identity matrix, which just makes that step behave like plain steepest descent again.

3. The App

Control panel — pick the method (steepest descent / Newton / BFGS / all three at once), pick the step-size rule (exact or Armijo), pick the test function, set the starting point, set the max number of steps and the tolerance (how small the gradient needs to get before we call it "converged"), and press Run, run the hardness study, or reset the view.

Contour plot — shows the shape of the chosen function as level curves, and draws the path each method took on top of it (steepest descent in red, Newton in orange, BFGS in purple).

- Convergence chart — shows how fast the gradient size shrinks over the iterations, on a log scale, with one line per method. A line that drops fast means that method is converging quickly.
- Condition-number chart — shows how many steps each method needs, as the function is made harder and harder (this is the "hardness study" mentioned above). Also on a log scale, so the different growth rates are easy to compare. If a line goes up as you move right, that method is struggling more as the function gets harder. If a line stays flat as you move right, that method doesn't care how hard the function gets, it just keeps solving it in about the same number of steps. The condition-number chart doesn't actually depend on which function you picked. It runs its own separate test in the background, using one stretched-out bowl shape, and just makes it harder and harder by increasing κ . This actually makes sense, since the whole point of the chart is to see how each method handles a problem that's slowly getting tougher, in a fair and consistent way. If it kept switching between different functions instead, the comparison wouldn't really make sense, because too many things would be changing at the same time. So it keeps everything else fixed and only increases the difficulty (κ). That's also why picking a different function has no effect on this chart, while changing the step-size method, the tolerance, or the max number of steps does, since those are the only things this particular test actually responds to.



3.1 Which class does what

File / class	Layer	Responsibility
Objective.h — IObjective, Quadratic2D, Rosenbrock, Himmelblau, ...	Model	Has the math for each test function: the function itself, its gradient, its Hessian, plus a default starting point for each one
Optimizer.h — Optimizer	Model	The main loop that all three methods run through. Picks the direction and keeps track of every step taken
LineSearch.h — LineSearch	Model	Figures out how big of a step to take, using either exact line search or Armijo backtracking
CondStudy.h — CondStudy	Model	Runs the optimizer over and over on harder and harder functions, to make the condition-number chart
ControlsView.h — ControlsView	GUI	The control panel on the left: pick the method, the function, the starting point, press Run...
ContourCanvas.h — ContourCanvas	GUI	Draws the contour plot with the paths on it
ChartCanvas.h — ChartCanvas	GUI	A simple line-chart tool, used for both the convergence chart and the condition-number chart
MainView.h — MainView	GUI	Puts the control panel and the three charts together into one screen
MainWindow.h / Application.h / main.cpp	GUI	Where the app starts, the main window, and the menu/toolbar buttons